

SMART PLACEMENT PREPARATION PLATFORM (PREPSMART)

A Project Report Submitted in Partial Fulfillment of the
Requirements for the Award of the Degree of

Bachelor of Technology

in

Computer Science and Engineering

Submitted by

Mala Neeraj Srinivas

Roll No.: UI21CSC35

Under the Guidance of

Industry Mentor

Akhilesh Reddy Karra

Architect - Application (AuroPro Soft Systems Pvt Ltd), Hyderabad

Faculty Supervisor

Dr. Kaustubh Dhondge

Faculty Examiner

Dr. Reema Patel



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY SURAT

May 2026

Surat – Gujarat – 394190

Indian Institute of Information Technology Surat
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that **Mala Neeraj Srinivas** (Roll No. **UI21CSC35**), a student of the B.Tech. Programme in the Department of Computer Science and Engineering, Indian Institute of Information Technology Surat, has successfully completed the major project entitled

“PrepSmart: Design and Development of a Smart Placement Preparation Platform”

under the academic supervision of **Dr. Kaustubh Dhondge** and industrial guidance of **Akhilesh Reddy Karra, Architect - Application (AuroPro Soft Systems Pvt Ltd), Hyderabad**, during the academic year 2025–2026.

This project report is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**.

Faculty Supervisor

Dr. Kaustubh Dhondge

Signature

Faculty Examiner

Dr. Reema Patel

Signature

Director, IIIT Surat

Signature

(Institute Seal)

DECLARATION

I, **Mala Neeraj Srinivas** (Roll No. **UI21CSC35**), hereby declare that the project report entitled

“PrepSmart: Design and Development of a Smart Placement Preparation Platform”

submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** at the **Indian Institute of Information Technology Surat**, is my original work carried out under the guidance of **Dr. Kaustubh Dhondge** and the industrial mentorship of **Akhilesh Reddy Karra**.

I further declare that this work has not been submitted, either in whole or in part, to any other university or institution for the award of any degree, diploma, or other academic qualification. Wherever the work of others has been used, appropriate acknowledgement has been made through proper citations and references.

Mala Neeraj Srinivas

Roll No.: UI21CSC35

B.Tech. IV Year, CSE

Indian Institute of Information Technology Surat

July, 2026

Signature: _____

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to all those who have contributed to the successful completion of this project and the preparation of this report.

I express my heartfelt gratitude to my Industry Mentor, **Akhilesh Reddy Karra**, Architect – Applications, at **AuroPro Soft Systems Pvt. Ltd.**, for his invaluable guidance, constant encouragement, technical expertise, and continuous support throughout the development of this project. His professional mentorship, insightful suggestions, and industry perspective greatly contributed to the successful completion of this work.

I am deeply grateful to my Faculty Supervisor, **Dr. Kaustubh Dhondge**, Associate Dean (Academics), for his constant encouragement, valuable guidance, and academic support throughout the course of this project. His mentorship played a significant role in the successful completion of this work.

I sincerely thank my Faculty Examiner, **Dr. Reema Patel**, Assistant Professor, for her valuable suggestions and constructive feedback, which helped improve the quality of this project.

I would also like to express my sincere appreciation to **Polukanti Ravi Kumar** and **Chitukuru Nalini** for their continuous support, motivation, and encouragement throughout this journey.

I extend my gratitude to the faculty and staff of the **Department of Computer Science and Engineering, Indian Institute of Information Technology Surat**, for providing an excellent academic environment and the resources necessary to undertake this project successfully.

Finally, I express my sincere gratitude to the **Director, Indian Institute of Information Technology Surat**, for providing the institutional support and opportunities that made this project possible.

Mala Neeraj Srinivas

Roll No.: UI21CSC35

B.Tech. IV Year, CSE

IIIT Surat

May 2026

ABSTRACT

PrepSmart: Design and Development of a Smart Placement Preparation Platform is a web-based application developed to provide a unified and structured environment for campus placement preparation. Students often depend on multiple independent platforms for learning, coding practice, mock tests, interview preparation, and progress tracking, resulting in fragmented learning and inefficient preparation. PrepSmart addresses this challenge by integrating these activities into a single platform that supports systematic and personalized learning.

The platform provides structured learning paths, coding practice, mock tests, AI-assisted interview preparation, company-specific preparation, resume and portfolio management, and performance analytics. These modules enable students to monitor their learning progress, identify weak areas, and prepare effectively for technical interviews and campus recruitment.

The system is implemented using **React** for the frontend, **Django** and **Django REST Framework (DRF)** for the backend, and **MySQL** as the database. Secure authentication is achieved using **JSON Web Tokens (JWT)**, while RESTful APIs enable efficient communication between the frontend and backend. The platform also includes an integrated coding workspace with code execution and evaluation support.

PrepSmart follows a modular and scalable architecture that simplifies maintenance and supports future enhancements such as adaptive learning recommendations, advanced AI-driven mentoring, and cloud deployment. By combining learning, assessment, coding practice, interview preparation, and analytics within a unified system, the platform enhances learning efficiency, improves placement readiness, and provides students with a comprehensive solution for campus recruitment preparation.

Keywords: Placement Preparation, React, Django, Django REST Framework, MySQL, JWT Authentication, REST API, Educational Technology.

Contents

Certificate	ii
Declaration	iii
Acknowledgements	iv
Abstract	v
List of Abbreviations and Acronyms	viii
List of Figures	x
List of Tables	x
1 Introduction	1
1.1 Introduction	1
1.2 Problem Statement	2
1.3 Objectives	3
1.4 Scope of the Project	4
1.5 Proposed Methodology	5
1.6 Organization of the Report	6
2 Literature Survey	7
2.1 Introduction	7
2.2 Review of Existing Placement Preparation Platforms	8
2.2.1 LeetCode	8
2.2.2 HackerRank	8
2.2.3 GeeksforGeeks	9
2.2.4 InterviewBit	9
2.2.5 LinkedIn Learning	9

2.3	Comparative Analysis of Existing Platforms	9
2.4	Research Gap	10
2.5	Chapter Summary	11
3	System Analysis and Design	12
3.1	Introduction	12
3.2	Requirement Analysis	12
3.2.1	Functional Requirements	13
3.2.2	Non-Functional Requirements	14
3.3	Feasibility Study	15
3.3.1	Technical Feasibility	15
3.3.2	Economic Feasibility	16
3.3.3	Operational Feasibility	16
3.4	Overall System Architecture	16
3.5	Five-Layer Software Architecture	18
3.5.1	Presentation Layer	19
3.5.2	API Layer	19
3.5.3	Business Logic Layer	19
3.5.4	Data Access Layer	19
3.5.5	External Service Layer	19
3.6	High-Level Software Architecture	20
3.6.1	Client Layer	21
3.6.2	Backend Layer	22
3.6.3	Data Layer	22
3.6.4	External Service Layer	22
3.7	Use Case Diagram	23
3.7.1	Guest User	25
3.7.2	Student User	25
3.7.3	Administrator	25
3.8	Data Flow Diagram	26
3.8.1	Working of the Data Flow	27
3.8.2	Advantages of the Data Flow Design	28
3.9	Class Diagram	28
3.9.1	Advantages of the Class Design	31

3.10 Database Design	31
3.10.1 Major Database Entities	32
3.10.2 Relationship Between Entities	33
3.10.3 Database Features	33
3.10.4 Advantages of the Database Design	34
3.11 Chapter Summary	34
4 System Development and Implementation	35
4.1 Introduction	35
4.2 Technology Stack	36
4.3 Development Environment	37
4.4 Frontend Implementation	39
4.4.1 Advantages of the Frontend Architecture	40
4.5 Backend Implementation	41
4.5.1 API Request Processing Workflow	42
4.5.2 Advantages of the Backend Architecture	42
4.6 Authentication Module	43
4.6.1 Authentication Workflow	44
4.6.2 Security Features	45
4.6.3 Advantages of the Authentication Module	45
4.7 Placement Preparation Module	46
4.7.1 Placement Preparation Workflow	47
4.7.2 Major Features	47
4.7.3 Advantages of the Placement Preparation Module	48
References	49

LIST OF ACRONYMS

Acronym	Description
AI	Artificial Intelligence
API	Application Programming Interface
ATS	Applicant Tracking System
CRUD	Create, Read, Update and Delete
CSS	Cascading Style Sheets
DBMS	Database Management System
DRF	Django REST Framework
ER	Entity Relationship
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
ORM	Object Relational Mapper
REST	Representational State Transfer
SPA	Single Page Application
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience
UUID	Universally Unique Identifier

List of Figures

3.1	Overall System Architecture of PrepSmart	17
3.2	Five-Layer Software Architecture of PrepSmart	18
3.3	High-Level Software Architecture of PrepSmart	21
3.4	Use Case Diagram of PrepSmart	24
3.5	Level-0 Data Flow Diagram of PrepSmart	26
3.6	Class Diagram of PrepSmart	29
3.7	Entity Relationship Diagram of PrepSmart	31
4.1	React Component Architecture of PrepSmart	39
4.2	API Request Flow in PrepSmart	41
4.3	Authentication Sequence Diagram of PrepSmart	43
4.4	Placement Preparation Workflow	46

List of Tables

2.1	Comparison of Existing Placement Preparation Platforms	10
3.1	Functional Requirements of PrepSmart	14
3.2	Non-Functional Requirements of PrepSmart	15
4.1	Technology Stack Used in PrepSmart	36
4.2	Development Environment of PrepSmart	38

Chapter 1

Introduction

1.1 Introduction

Campus placement is one of the most important milestones in a student's academic journey as it marks the transition from higher education to the professional world. In today's competitive recruitment environment, organizations expect graduates to possess strong programming skills, sound knowledge of core computer science subjects, analytical thinking, communication abilities, practical project experience, and problem-solving capabilities. Consequently, placement preparation has become a continuous process that requires students to develop multiple competencies simultaneously rather than focusing solely on academic performance.

With the rapid growth of online learning platforms, students now have access to a vast collection of educational resources for coding practice, aptitude preparation, interview guidance, resume building, and technical learning. Although these platforms have improved accessibility to learning materials, they generally focus on individual aspects of placement preparation and operate independently. Students are therefore required to switch between multiple applications throughout their preparation journey, making it difficult to maintain consistency, monitor progress, and evaluate their overall readiness for campus recruitment.

Most existing preparation methods lack a unified mechanism for combining learning, assessments, coding practice, interview preparation, and analytical feedback within a single environment. As a result, students often spend considerable time managing different resources instead of concentrating on continuous learning and skill improvement. Furthermore, the absence of centralized progress tracking limits their ability to identify weak areas and plan effective preparation strategies.

To address these challenges, this project presents **PrepSmart: Design and Development of a Smart Placement Preparation Platform**, a comprehensive web-based application that integrates the essential components of placement preparation into a single ecosystem. The platform provides structured learning paths,

topic-wise assessments, coding practice, AI-assisted interview preparation, company-specific preparation, resume and portfolio management, personalized dashboards, and analytical performance tracking. By consolidating these functionalities into one platform, PrepSmart enables students to prepare in a structured, measurable, and efficient manner.

The application is developed using modern full-stack technologies, with React for the frontend, Django and Django REST Framework (DRF) for backend development, and MySQL as the database management system. Secure authentication is implemented using JSON Web Tokens (JWT), while RESTful APIs facilitate seamless communication between different components of the system. The modular architecture adopted by the platform ensures scalability, maintainability, and flexibility for future enhancements.

Overall, PrepSmart aims to simplify the placement preparation process by providing students with an integrated platform that supports learning, practice, assessment, performance analysis, and career development. The proposed system improves preparation efficiency, reduces dependency on multiple applications, and provides continuous analytical feedback to enhance students' employability and placement readiness.

1.2 Problem Statement

The placement preparation process has become increasingly complex due to the growing expectations of recruiters and the availability of numerous independent learning resources. Students are expected to prepare for aptitude tests, programming assessments, technical interviews, behavioral interviews, resume screening, and company-specific recruitment processes within a limited period. However, the absence of a unified preparation platform often forces students to depend on multiple applications and websites to accomplish these tasks.

Existing platforms generally specialize in only one aspect of placement preparation. Coding platforms focus on programming practice, learning platforms provide theoretical content, interview platforms emphasize interview experiences, and resume builders assist in resume creation. Since these services operate independently, students must repeatedly switch between platforms, resulting in fragmented learning, inconsistent preparation strategies, and difficulty in monitoring their overall progress.

Another major limitation is the lack of comprehensive performance analytics. Most available systems provide isolated metrics such as coding scores or quiz results but fail to present a consolidated view of a student's placement readiness. Consequently, students often find it difficult to identify weak areas, evaluate their preparation status, and prioritize topics requiring additional attention.

Furthermore, many existing systems offer limited personalization and do not effectively support company-specific preparation. Students preparing for different organizations often need to manually collect interview experiences, recruitment patterns, aptitude questions, and technical resources from multiple sources, making the preparation process inefficient and time-consuming.

Therefore, there is a need for a centralized, intelligent, and scalable platform that integrates learning resources, coding practice, assessments, interview preparation, resume management, portfolio development, and analytical dashboards within a single application. Such a platform should enable students to prepare systematically, continuously monitor their progress, and make data-driven improvements throughout their placement journey. PrepSmart has been developed to address these challenges by providing an integrated ecosystem for comprehensive placement preparation.

1.3 Objectives

The primary objective of PrepSmart is to design and develop a comprehensive web-based placement preparation platform that integrates learning, assessment, coding practice, interview preparation, and performance analysis within a single application. The platform aims to simplify the placement preparation process while enabling students to continuously monitor their progress and improve their overall employability.

The specific objectives of the proposed system are as follows:

1. To develop a centralized platform that combines various placement preparation activities into a unified environment.
2. To provide structured learning paths covering aptitude, programming, computer science fundamentals, interview preparation, and company-specific learning resources.
3. To implement coding practice with an integrated code editor and execution support for improving programming skills.
4. To design and develop mock tests, quizzes, and assessment modules for continuous evaluation of student performance.
5. To provide AI-assisted interview preparation that enables students to practice interview scenarios and improve their communication and technical skills.
6. To implement secure user authentication and profile management using JSON Web Tokens (JWT).

-
7. To provide personalized dashboards that present learning progress, coding performance, assessment scores, and placement readiness through analytical visualizations.
 8. To maintain resume and portfolio information within the platform, allowing students to organize their professional profiles for placement activities.
 9. To adopt a modular software architecture that supports scalability, maintainability, and future enhancements.
 10. To develop a user-friendly, responsive, and interactive web application using modern full-stack technologies.

1.4 Scope of the Project

PrepSmart is designed as a comprehensive web-based placement preparation platform that integrates multiple activities involved in campus recruitment into a single application. The project focuses on simplifying the preparation process by providing structured learning resources, coding practice, assessments, interview preparation, and performance analysis through a unified and user-friendly interface.

The scope of the project is categorized as follows:

Functional Scope

The platform provides the following core functionalities:

- User registration, login, and profile management.
- Structured learning paths for placement preparation.
- Topic-wise quizzes and mock tests.
- Integrated coding workspace with code execution support.
- AI-assisted interview preparation.
- Company-specific preparation modules.
- Resume and portfolio management.
- Personalized dashboard with progress analytics.
- Daily goals and preparation tracking.

Technical Scope

The system is developed as a modern full-stack web application using React for the frontend and Django with Django REST Framework (DRF) for backend services. RESTful APIs facilitate communication between system components, while JSON Web Tokens (JWT) provide secure authentication and authorization. The modular architecture enables scalability, maintainability, and easy integration of future features.

Future Scope

Although the current implementation provides a complete placement preparation environment, the platform has been designed to support future enhancements. Potential improvements include:

- AI-based personalized learning recommendations.
- Cloud deployment for improved scalability.
- Support for multiple programming languages in the coding environment.
- Integration with online coding platforms and recruitment portals.
- Mobile application support.
- Advanced analytics and recruiter dashboards.

1.5 Proposed Methodology

The development of PrepSmart follows a structured Software Development Life Cycle (SDLC) approach to ensure systematic planning, design, implementation, testing, and deployment. The methodology adopted for the project consists of the following phases:

1. **Requirement Analysis:** The functional and non-functional requirements were identified through an analysis of existing placement preparation platforms and discussions with students to understand the challenges faced during placement preparation.
2. **System Design:** Based on the identified requirements, the overall software architecture, database schema, module structure, user interfaces, and API design were planned. UML diagrams and database models were prepared to provide a clear blueprint for implementation.

-
3. **Implementation:** The frontend was developed using React to provide a responsive and interactive user interface, while the backend was implemented using Django and Django REST Framework (DRF). RESTful APIs were developed to facilitate communication between the frontend and backend. JSON Web Tokens (JWT) were used to implement secure authentication and authorization.
 4. **Database Development:** The database was designed to efficiently manage user profiles, learning resources, coding activities, assessments, interview records, and analytical data while ensuring data integrity and scalability.
 5. **Integration and Testing:** Individual modules were integrated into a unified system and tested using unit testing, integration testing, and functional testing to ensure correctness, reliability, and seamless interaction among system components.
 6. **Evaluation:** The completed platform was evaluated by verifying its core functionalities, user experience, security, and overall performance to ensure that the project objectives were successfully achieved.

1.6 Organization of the Report

This report is organized into seven chapters. Chapter 1 introduces the project, defines the problem statement, presents the objectives, scope, and development methodology. Chapter 2 reviews existing literature and analyzes current placement preparation platforms to identify research gaps. Chapter 3 discusses the system analysis and design, including requirement analysis, software architecture, database design, and UML diagrams. Chapter 4 presents the implementation details of the proposed system, describing the technologies used and the development of individual modules. Chapter 5 discusses the experimental results, system features, and performance analysis. Chapter 6 presents the testing strategy, validation techniques, and test results. Finally, Chapter 7 summarizes the project, highlights its contributions, discusses limitations, and outlines possible future enhancements.

Chapter 2

Literature Survey

2.1 Introduction

The literature survey is an essential phase of software development that involves studying existing systems, research contributions, and technologies related to the proposed project. It helps in understanding current approaches, identifying their strengths and limitations, and recognizing the research gap that motivates the development of a new solution. For the PrepSmart platform, the literature survey focuses on modern placement preparation platforms, online learning systems, coding practice portals, interview preparation resources, and educational technologies that support campus recruitment.

Over the past decade, digital learning platforms have significantly transformed placement preparation by providing students with online coding environments, structured learning materials, mock assessments, interview experiences, and skill development resources. These platforms have improved accessibility to quality educational content and enabled students to prepare independently from any location. However, most existing systems focus on specific aspects of placement preparation rather than providing a unified learning ecosystem.

Students preparing for campus recruitment typically rely on multiple platforms for coding practice, aptitude preparation, interview guidance, resume development, and progress tracking. This fragmented approach often results in inconsistent preparation, inefficient resource management, and difficulty in evaluating overall placement readiness. Furthermore, many platforms provide limited personalization and lack integrated analytical features that help students identify strengths and weaknesses throughout their preparation journey.

The objective of this chapter is to examine widely used placement preparation platforms, analyze their features, identify their limitations, and establish the necessity for a comprehensive platform such as PrepSmart. The findings of this survey provide

the foundation for the design and development decisions presented in the subsequent chapters.

2.2 Review of Existing Placement Preparation Platforms

The increasing demand for technical skills has resulted in the emergence of several on-line platforms that support placement preparation. These platforms provide resources such as programming challenges, interview experiences, technical articles, aptitude questions, and online courses. Although they have significantly improved accessibility to educational content, most of them focus on specific aspects of placement preparation rather than providing a comprehensive solution. The following subsections present a review of widely used placement preparation platforms.

2.2.1 LeetCode

LeetCode is one of the most widely used online coding platforms for technical interview preparation. It provides a large collection of programming problems covering data structures, algorithms, databases, system design, and competitive programming. The platform also offers company-specific interview questions, coding contests, and discussion forums that help students prepare for software engineering interviews.

Despite its extensive collection of coding problems, LeetCode primarily focuses on programming practice and interview questions. It does not provide aptitude preparation, resume management, personalized learning paths, interview simulations, or integrated placement analytics, requiring students to depend on additional platforms for complete preparation.

2.2.2 HackerRank

HackerRank is an online platform designed for programming practice, skill assessment, and technical recruitment. It supports multiple programming languages and provides challenges related to algorithms, databases, artificial intelligence, and software development. Organizations also use HackerRank to conduct online coding assessments during recruitment.

Although HackerRank offers an excellent environment for coding practice and technical assessments, it provides limited support for interview preparation, learning progress analysis, resume management, and company-specific preparation. Students must therefore combine HackerRank with other learning resources to prepare comprehensively.

2.2.3 GeeksforGeeks

GeeksforGeeks is a popular educational platform that provides tutorials, interview preparation articles, coding questions, aptitude resources, and explanations of computer science concepts. It serves as a valuable reference for students preparing for technical interviews by offering extensive theoretical content and programming tutorials.

However, GeeksforGeeks primarily functions as a learning resource rather than an integrated placement preparation platform. Features such as personalized dashboards, structured preparation tracking, AI-assisted interview practice, and unified progress monitoring are limited.

2.2.4 InterviewBit

InterviewBit focuses on technical interview preparation through guided learning paths, coding exercises, and interview-oriented programming questions. The platform emphasizes structured learning and provides company-specific interview experiences that assist students in understanding real recruitment processes.

While InterviewBit offers organized learning content and coding practice, it lacks several complementary features such as resume management, portfolio development, AI-assisted interviews, integrated analytics, and comprehensive placement tracking.

2.2.5 LinkedIn Learning

LinkedIn Learning is an online educational platform offering professional courses across technology, business, communication, and leadership domains. Students can access expert-led video courses to strengthen both technical and soft skills required for career development.

Although LinkedIn Learning provides high-quality educational content, it is not specifically designed for placement preparation. It does not include integrated coding practice, mock assessments, company-specific preparation, or placement performance analytics within a unified platform.

2.3 Comparative Analysis of Existing Platforms

Table 2.1 presents a comparative analysis of widely used placement preparation platforms based on important features required during campus recruitment. The comparison highlights that while existing platforms perform well in specific domains such as coding practice or technical learning, none of them provide a unified ecosystem

that integrates learning, assessments, coding practice, interview preparation, resume management, and analytical performance tracking within a single application.

Table 2.1: Comparison of Existing Placement Preparation Platforms

Feature	LeetCode	HackerRank	GFG	InterviewBit	LinkedIn Learning	PrepSmart
Coding Practice	✓	✓	✓	✓	✗	✓
Structured Learning Paths	✗	Limited	✓	✓	✓	✓
Aptitude Preparation	✗	Limited	✓	Limited	✗	✓
Computer Science Subjects	Limited	Limited	✓	✓	✓	✓
Mock Tests	✗	✓	Limited	✓	✗	✓
AI Interview Preparation	✗	✗	✗	✗	✗	✓
Resume Management	✗	✗	✗	✗	✗	✓
Portfolio Management	✗	✗	✗	✗	✗	✓
Company-Specific Preparation	Limited	Limited	Limited	✓	✗	✓
Performance Analytics	Limited	Limited	Limited	Limited	✗	✓
Progress Dashboard	✗	Limited	✗	Limited	✗	✓
Single Integrated Platform	✗	✗	✗	✗	✗	✓

2.4 Research Gap

The comparative analysis demonstrates that existing placement preparation platforms primarily focus on individual aspects of learning rather than providing a comprehensive preparation environment. Coding platforms such as LeetCode and HackerRank emphasize programming practice, while educational platforms such as GeeksforGeeks and LinkedIn Learning concentrate on theoretical learning and professional development. InterviewBit offers structured interview preparation but lacks several complementary features required for complete placement readiness.

A significant limitation observed across these platforms is the absence of an integrated ecosystem that combines structured learning, coding practice, aptitude preparation, interview preparation, resume management, portfolio development, company-specific preparation, and analytical dashboards within a single application. Students are therefore required to use multiple independent platforms, resulting in fragmented learning experiences, inefficient progress tracking, and inconsistent preparation strategies.

Another important research gap is the limited use of intelligent analytics for monitoring placement readiness. Existing systems generally provide isolated performance metrics without offering comprehensive insights into learning progress, topic-wise strengths and weaknesses, or overall employability. Similarly, AI-assisted interview preparation and unified career management features remain largely unavailable across existing platforms.

The identified research gap establishes the need for a comprehensive and intelligent placement preparation platform capable of integrating multiple preparation

activities into a single ecosystem. PrepSmart has been developed to address these limitations by providing a unified platform that combines learning resources, coding practice, assessments, interview preparation, resume and portfolio management, company-specific preparation, and personalized performance analytics.

2.5 Chapter Summary

This chapter presented a detailed review of existing placement preparation platforms and analyzed their strengths and limitations. A comparative study highlighted that although platforms such as LeetCode, HackerRank, GeeksforGeeks, InterviewBit, and LinkedIn Learning provide valuable learning resources, they primarily focus on specific aspects of placement preparation rather than offering a comprehensive solution.

The analysis identified the absence of a unified platform capable of integrating structured learning, coding practice, assessments, AI-assisted interview preparation, resume management, portfolio development, company-specific preparation, and performance analytics within a single ecosystem. These observations establish the motivation for developing PrepSmart as an integrated and scalable placement preparation platform.

The next chapter presents the system analysis and design of the proposed solution, including requirement analysis, software architecture, database design, UML diagrams, and overall system workflow.

Chapter 3

System Analysis and Design

3.1 Introduction

System analysis and design is one of the most important phases in the Software Development Life Cycle (SDLC), as it establishes the foundation for developing a reliable, scalable, and maintainable software system. During this phase, the requirements of the proposed system are analyzed, the overall architecture is designed, and the interactions between different system components are defined before implementation begins.

For the PrepSmart platform, the analysis phase focused on understanding the challenges faced by students during placement preparation and identifying the functional requirements necessary to address those challenges. Existing placement preparation platforms were studied to determine their strengths and limitations, enabling the design of a unified solution that integrates multiple preparation activities within a single application.

Based on the identified requirements, the system architecture, software modules, database structure, and user interactions were carefully designed. The platform follows a modular architecture in which each module performs a specific responsibility while remaining seamlessly integrated with the overall system. This design approach improves maintainability, scalability, and future extensibility.

This chapter presents the requirement analysis, feasibility study, software architecture, UML models, database design, and overall system design adopted during the development of PrepSmart. These design artifacts provide a clear blueprint for the implementation discussed in the subsequent chapter.

3.2 Requirement Analysis

Requirement analysis is the process of identifying and documenting the functional and non-functional requirements of a software system. It ensures that the developed

application satisfies user expectations while maintaining reliability, security, scalability, and performance. The requirements for PrepSmart were identified by studying existing placement preparation platforms, understanding the campus recruitment process, and analyzing the practical challenges encountered by students during placement preparation.

The identified requirements were classified into two major categories:

- Functional Requirements
- Non-Functional Requirements

3.2.1 Functional Requirements

Functional requirements describe the services and operations that the system must perform to satisfy user needs. These requirements define the core functionalities implemented in PrepSmart and determine how users interact with different modules of the platform.

The major functional requirements of the proposed system are summarized in Table 3.1.

Table 3.1: Functional Requirements of PrepSmart

ID	Requirement	Description
FR1	User Authentication	The system shall allow users to register, log in, and securely authenticate using JWT.
FR2	User Profile Management	Users shall be able to manage their personal and academic information.
FR3	Learning Paths	The platform shall provide structured learning paths for placement preparation.
FR4	Coding Practice	Users shall be able to solve programming problems using the integrated coding environment.
FR5	Mock Tests	The system shall provide quizzes and mock tests for continuous assessment.
FR6	AI Interview Module	Users shall be able to practice technical interviews using AI-assisted interview sessions.
FR7	Resume Management	The platform shall allow users to create and manage resumes.
FR8	Portfolio Management	Users shall be able to organize projects, skills, and achievements.
FR9	Company Preparation	The system shall provide company-specific preparation resources and progress tracking.
FR10	Dashboard Analytics	Users shall be able to monitor learning progress and placement readiness through personalized dashboards.

3.2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes of the proposed system. These requirements ensure that PrepSmart is secure, reliable, scalable, efficient, and easy to maintain.

The major non-functional requirements are summarized in Table 3.2.

Table 3.2: Non-Functional Requirements of PrepSmart

Requirement	Description
Performance	The system should respond efficiently to user requests with minimal latency.
Security	Authentication and authorization shall be implemented using JWT to protect user data.
Scalability	The architecture should support future modules and increased numbers of users.
Reliability	The platform should provide consistent and dependable operation under normal usage.
Usability	The user interface should be simple, intuitive, and responsive across different devices.
Maintainability	The modular software architecture should simplify future updates and maintenance.
Availability	The platform should remain accessible whenever users require placement preparation resources.
Portability	The application should be deployable across different operating systems and hosting environments.

3.3 Feasibility Study

A feasibility study is conducted to evaluate whether the proposed system can be successfully developed and deployed within the available resources, time, and technical constraints. It helps determine the practicality and viability of the project before implementation. For the PrepSmart platform, the feasibility analysis is classified into technical, economic, and operational feasibility.

3.3.1 Technical Feasibility

The technical feasibility of PrepSmart is high because the project is developed using widely adopted open-source technologies. The frontend is implemented using React, while the backend is developed using Django and Django REST Framework (DRF). The database is managed using MySQL, and secure authentication is provided through JSON Web Tokens (JWT). These technologies are stable, well-documented, and supported by large developer communities, making the system reliable and maintainable.

The development environment, including Visual Studio Code, Git, GitHub, and Postman, provides sufficient support for implementation, testing, and version con-

trol. Therefore, the proposed system can be developed efficiently without requiring specialized hardware or proprietary software.

3.3.2 Economic Feasibility

The development cost of PrepSmart is minimal because the project primarily utilizes free and open-source software tools. The programming languages, frameworks, database management system, and development utilities are freely available, eliminating licensing expenses.

The hardware requirements are limited to a standard personal computer capable of running the development environment. Consequently, the overall cost of developing and maintaining the system remains low, making the project economically feasible for educational institutions and individual developers.

3.3.3 Operational Feasibility

PrepSmart is designed with a user-friendly interface that enables students to access various placement preparation resources through a single platform. The system reduces the need to switch between multiple applications by integrating learning paths, coding practice, mock tests, interview preparation, resume management, and analytical dashboards within one environment.

The modular architecture simplifies maintenance and allows future enhancements without affecting existing functionality. Since the platform aligns with the requirements of students preparing for campus placements, it is expected to improve preparation efficiency and enhance the overall user experience.

Based on the technical, economic, and operational analyses, the proposed system is considered feasible for development and deployment.

3.4 Overall System Architecture

The overall system architecture of PrepSmart follows a modular client–server architecture that separates the presentation layer, application logic, data management, and external service integrations into independent components. This separation of concerns improves maintainability, scalability, security, and simplifies future enhancements.

The architecture has been designed to provide efficient communication between the frontend and backend while allowing additional services such as AI-assisted interview evaluation, coding execution, and contest synchronization to be integrated without affecting the core application.

Figure 3.1 illustrates the overall architecture of the PrepSmart platform.

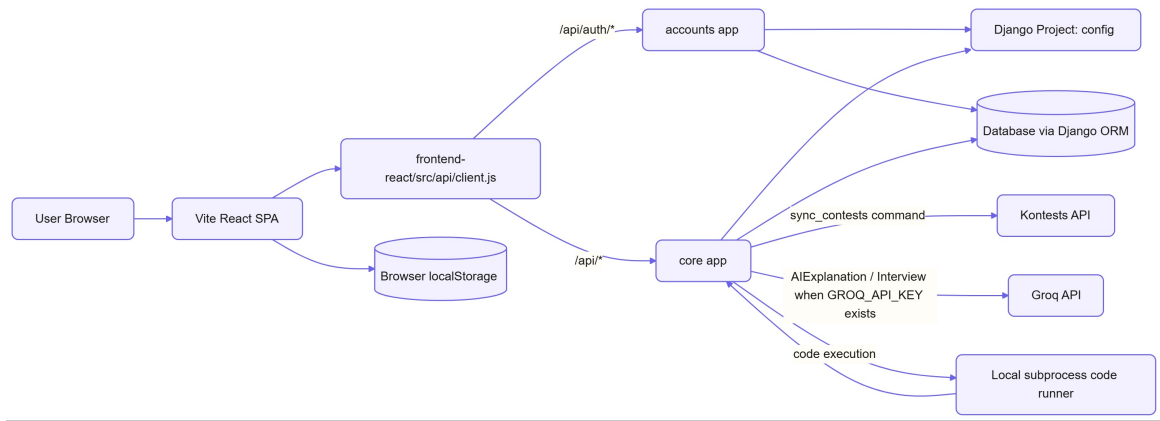


Figure 3.1: Overall System Architecture of PrepSmart

The architecture consists of four major components: the client layer, the application layer, the data layer, and external service integrations.

The client layer is represented by the user's web browser, which executes the React-based Single Page Application (SPA). The frontend provides an interactive user interface for learning paths, coding practice, mock tests, interview preparation, dashboards, and portfolio management. Since the application is implemented as a Single Page Application, navigation between pages occurs dynamically without requiring complete page reloads.

Communication between the frontend and backend is performed through a centralized API client. This component is responsible for sending HTTP requests, attaching JSON Web Token (JWT) authentication headers, handling server responses, refreshing expired access tokens, and maintaining consistent communication with backend services.

The backend is implemented using Django and Django REST Framework (DRF). It is logically divided into two applications. The Accounts application manages authentication, authorization, user profiles, and session management, whereas the Core application implements the primary business logic of PrepSmart, including learning management, coding practice, dashboards, interview preparation, company readiness analysis, and portfolio generation.

Persistent data storage is handled using Django ORM, which maps Python models to relational database tables. The ORM provides database abstraction, ensuring portability, improved security, and simplified database operations. User information, assessments, interview sessions, coding submissions, analytical reports, and placement preparation records are stored and managed through this layer.

The platform also integrates several external services to extend its functionality. The Kontests API is used to synchronize upcoming programming contests from

multiple competitive coding platforms. The Groq API provides AI-powered interview feedback and intelligent explanations whenever the corresponding API key is configured. Additionally, a local subprocess-based code execution engine compiles and executes user-submitted programs in a controlled environment before returning execution results to the frontend.

This modular architecture enables each component to evolve independently while maintaining seamless communication with other system modules. The resulting design improves reliability, maintainability, scalability, and facilitates future feature expansion.

3.5 Five-Layer Software Architecture

PrepSmart follows a five-layer software architecture that separates the system into independent layers based on their responsibilities. Each layer performs a specific set of functions and communicates only with adjacent layers through well-defined interfaces. This layered approach improves modularity, maintainability, scalability, and simplifies future enhancements by reducing dependencies between different components of the system.

The layered architecture enables frontend development, backend development, database management, and external service integration to evolve independently without affecting the overall functionality of the platform.

Figure 3.2 illustrates the five-layer architecture adopted for the PrepSmart platform.

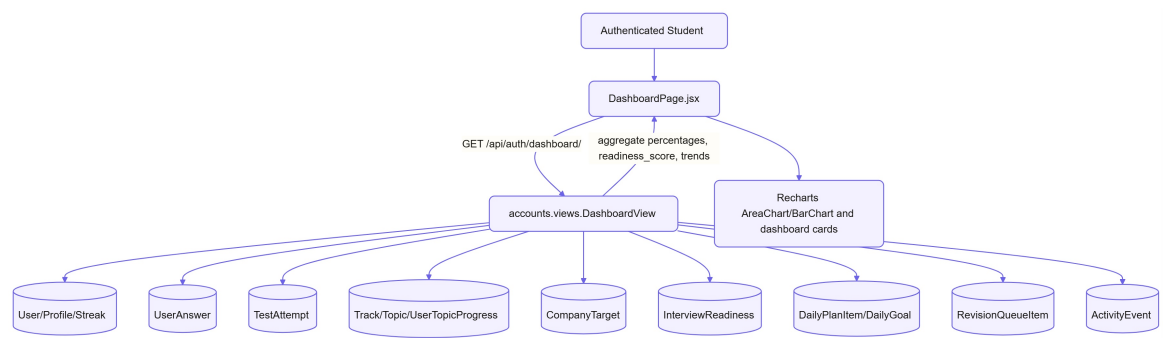


Figure 3.2: Five-Layer Software Architecture of PrepSmart

The architecture is organized into five logical layers, namely the Presentation Layer, API Layer, Business Logic Layer, Data Access Layer, and External Service Layer. Each layer is responsible for a specific functionality and communicates with the neighboring layers through standardized interfaces.

3.5.1 Presentation Layer

The Presentation Layer consists of the React-based Single Page Application (SPA), which provides an interactive and responsive user interface for students. This layer handles page navigation, user interaction, form validation, state management, and visualization of placement preparation data. Users interact with all modules of PrepSmart through this layer without directly accessing backend services.

3.5.2 API Layer

The API Layer acts as the communication bridge between the frontend and backend. It consists of RESTful APIs developed using Django REST Framework (DRF). All requests from the frontend, including authentication, dashboard access, coding submissions, mock tests, and interview sessions, are routed through this layer. The API Layer also validates authentication tokens and returns standardized JSON responses.

3.5.3 Business Logic Layer

The Business Logic Layer implements the core functionality of PrepSmart. This layer processes user requests, manages placement preparation workflows, evaluates coding submissions, tracks learning progress, generates analytical reports, and coordinates AI-assisted interview sessions. Separating business logic from the presentation layer improves code reusability and maintainability.

3.5.4 Data Access Layer

The Data Access Layer is implemented using Django ORM, which provides an abstraction between the application and the underlying relational database. Instead of executing raw SQL queries, the application performs database operations through ORM models. This improves portability, enhances security against SQL injection attacks, and simplifies database management.

3.5.5 External Service Layer

The External Service Layer integrates third-party services that extend the capabilities of the platform. The Kontests API provides information about upcoming programming contests, the Groq API enables AI-assisted interview evaluation and explanations, and the local code execution engine securely compiles and executes programming solutions submitted by users. Keeping these integrations separate from the core application improves modularity and simplifies future enhancements.

The layered architecture adopted in PrepSmart offers several advantages:

-
- Clear separation of responsibilities among software components.
 - Improved modularity and maintainability.
 - Easier debugging and testing of individual layers.
 - Enhanced scalability through independent development of each layer.
 - Improved security by restricting direct access to business logic and database operations.
 - Simplified integration of new external services without affecting existing modules.

3.6 High-Level Software Architecture

The high-level software architecture of PrepSmart illustrates the major software components of the system and the communication among them. The architecture follows a distributed client-server model, where the frontend, backend, database, and external services operate as independent modules while communicating through RESTful APIs.

The architecture has been designed to ensure loose coupling between system components, thereby improving maintainability, scalability, security, and ease of deployment. Each layer performs a well-defined responsibility, allowing future enhancements to be incorporated without significantly affecting existing modules.

Figure 3.3 presents the high-level software architecture adopted in the PrepSmart platform.

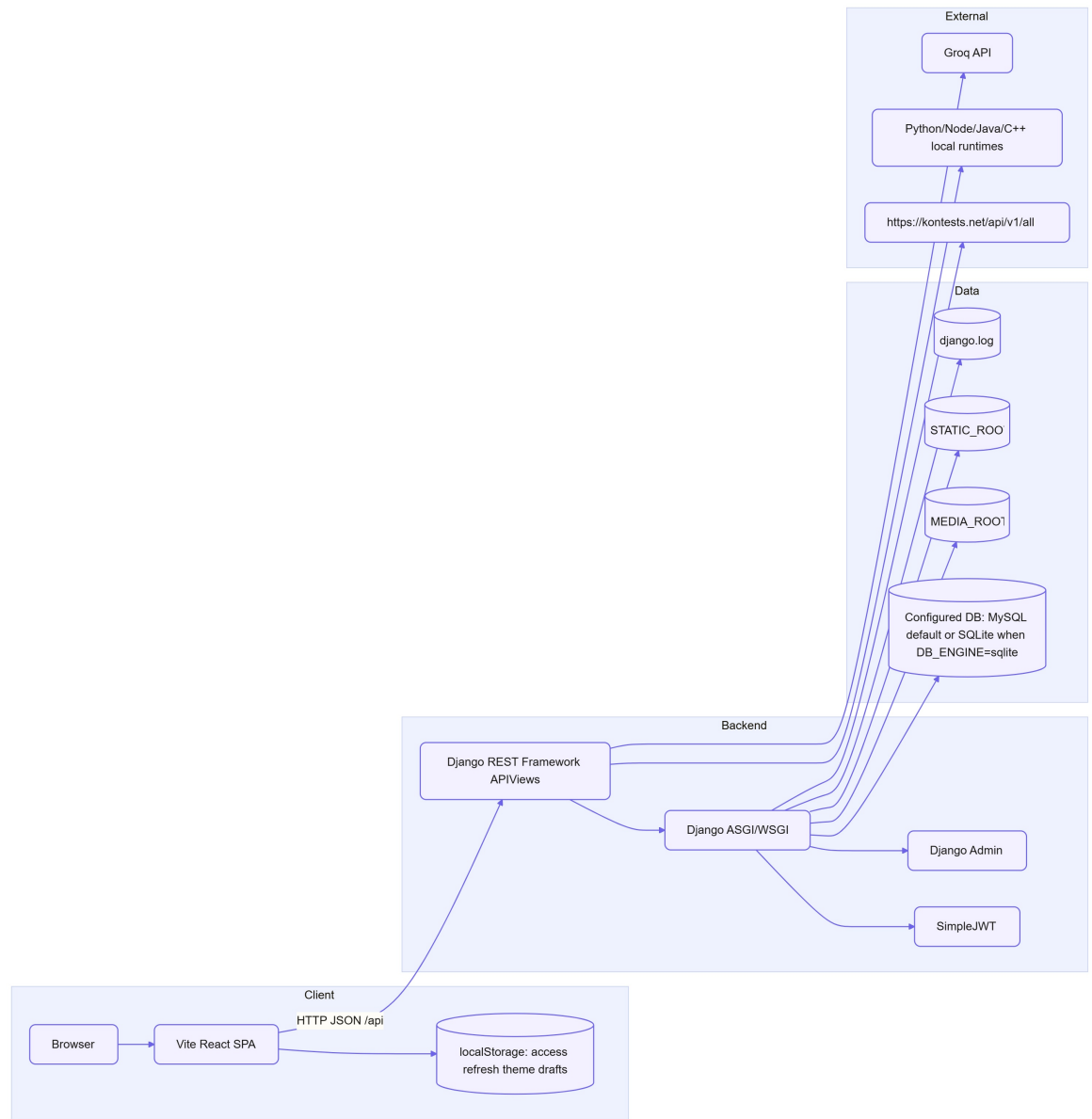


Figure 3.3: High-Level Software Architecture of PrepSmart

The software architecture is organized into four primary subsystems: the Client Layer, Backend Layer, Data Layer, and External Service Layer. These subsystems cooperate to provide a secure, responsive, and scalable placement preparation platform.

3.6.1 Client Layer

The Client Layer consists of the user's web browser and the React-based Single Page Application (SPA). It provides an interactive graphical user interface through which users perform activities such as authentication, topic learning, coding practice, mock tests, interview preparation, dashboard visualization, resume creation, and portfolio management. User preferences, authentication tokens, and temporary application

data are stored in the browser’s local storage to improve responsiveness and maintain user sessions.

3.6.2 Backend Layer

The Backend Layer is implemented using Django and Django REST Framework (DRF). It exposes RESTful API endpoints that process incoming requests from the frontend, enforce authentication and authorization, execute business logic, and generate JSON responses. The backend is further divided into separate applications, including the Accounts module for authentication and user management, and the Core module for placement preparation functionalities.

3.6.3 Data Layer

The Data Layer is responsible for persistent storage of application data. Django ORM provides an abstraction between the backend and the relational database, allowing models to be mapped directly to database tables. The database stores user accounts, learning progress, coding submissions, interview sessions, company preparation records, mock test results, dashboards, resumes, portfolios, and other application data.

Static files, uploaded media, and application logs are also maintained separately to improve organization and simplify deployment.

3.6.4 External Service Layer

The External Service Layer extends the functionality of PrepSmart by integrating third-party services. The Kontests API provides live information about upcoming programming contests, while the Groq API generates AI-assisted interview questions, explanations, and feedback whenever AI functionality is enabled. The local code execution environment compiles and executes user-submitted programs securely, returning execution results to the backend for evaluation and display.

The modular organization of these services allows external integrations to be modified or replaced without affecting the internal architecture of the platform.

The proposed high-level software architecture provides several benefits:

- Separation of presentation, business logic, data management, and external services.
- Improved scalability through independent software modules.
- Enhanced maintainability due to modular development practices.

-
- Simplified deployment and future system upgrades.
 - Secure communication using authenticated RESTful APIs.
 - Easy integration of AI services and external APIs without modifying core application logic.

3.7 Use Case Diagram

A Use Case Diagram is a behavioral UML diagram that illustrates the interactions between different users (actors) and the functionalities provided by the system. It provides a high-level view of the system from the user's perspective and helps identify the major services offered by the application.

The PrepSmart platform supports multiple categories of users, each having different roles and privileges. The primary actors are the Guest User, Student User, and Administrator. Each actor interacts with specific modules according to the permissions assigned within the system.

Figure 3.4 presents the use case diagram of the PrepSmart platform.

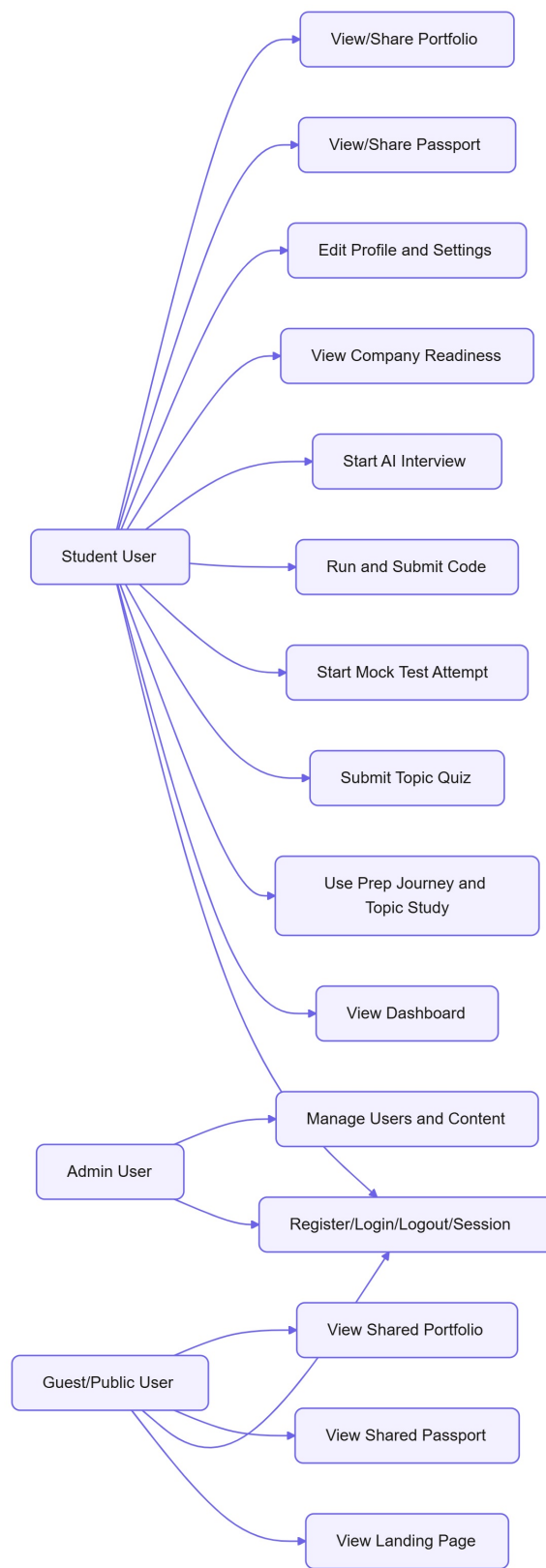


Figure 3.4: Use Case Diagram of PrepSmart

The use case diagram illustrates the interactions between the three major actors and the functionalities available within the PrepSmart platform.

3.7.1 Guest User

A Guest User can access the public landing page of the application and explore limited publicly available information. Guests can also view publicly shared portfolios and digital passports generated by registered users. To access personalized placement preparation services, the guest must complete the registration process and authenticate into the system.

3.7.2 Student User

The Student User is the primary actor of the system and has access to all placement preparation features after successful authentication. Students can manage their personal profiles, follow structured learning paths, solve coding problems, participate in mock tests, attend AI-assisted interview sessions, monitor their preparation progress through analytical dashboards, prepare for company-specific recruitment processes, generate resumes and portfolios, and share their achievements using public portfolio links.

The platform continuously tracks student activities and updates their progress automatically, enabling personalized recommendations and readiness analysis.

3.7.3 Administrator

The Administrator manages the overall operation of the platform. Administrative responsibilities include managing users, maintaining learning resources, monitoring coding problems, updating placement content, supervising company preparation modules, and ensuring the smooth functioning of backend services. Administrative users also monitor system activity and maintain data consistency throughout the platform.

The major use cases supported by PrepSmart include:

- User Registration and Login
- User Profile Management
- Dashboard Visualization
- Structured Learning Paths
- Topic-wise Quiz Practice
- Mock Test Participation
- Coding Practice and Code Execution

- AI-based Interview Preparation
- Company Readiness Analysis
- Resume Generation
- Portfolio Management
- Public Portfolio and Passport Sharing
- Administrative Content Management

The use case model provides several advantages during software development:

- Clearly identifies different categories of system users.
- Defines the responsibilities and permissions of each actor.
- Simplifies requirement validation during system analysis.
- Serves as a foundation for designing application workflows.
- Improves communication between developers and stakeholders.

3.8 Data Flow Diagram

A Data Flow Diagram (DFD) illustrates how information flows between different components of a software system. Unlike architectural diagrams that describe the structural organization of software components, a DFD focuses on the movement of data between external entities, system processes, data stores, and external services.

The Level-0 Data Flow Diagram of PrepSmart presents the overall interaction between the student, the React frontend, the Django REST backend, the relational database, and external services used by the platform.

Figure 3.5 illustrates the Level-0 Data Flow Diagram of the proposed system.

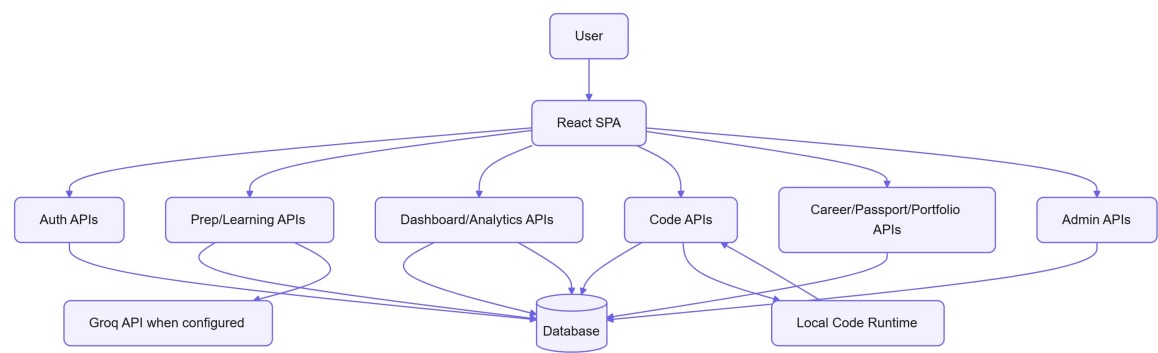


Figure 3.5: Level-0 Data Flow Diagram of PrepSmart

The Level-0 Data Flow Diagram consists of five major components, namely the User, React Single Page Application, Django REST API, Database, and External Services.

The User represents the primary external entity interacting with the system. Students perform activities such as registration, authentication, coding practice, mock tests, interview preparation, dashboard analysis, resume generation, and portfolio management through the user interface.

The React Single Page Application acts as the presentation layer of the system. It receives user inputs, performs client-side validation, manages application state, and communicates with the backend through RESTful API requests. The frontend never communicates directly with the database, ensuring proper separation of concerns and improved security.

The Django REST API serves as the central processing component of the system. It authenticates users using JSON Web Tokens (JWT), validates incoming requests, executes business logic, interacts with the database using Django ORM, and returns structured JSON responses to the frontend. Nearly all business operations within PrepSmart are coordinated through this layer.

The Database stores all persistent information required by the platform, including user profiles, learning progress, coding submissions, mock test attempts, interview sessions, portfolios, resumes, dashboards, company preparation records, and analytical data. Django ORM manages all database interactions, eliminating the need for direct SQL queries.

The External Services component includes the Groq API for AI-assisted interview feedback, the Kontests API for synchronizing programming contests, and the local code execution engine for compiling and evaluating user-submitted programming solutions. These services extend the capabilities of PrepSmart while remaining loosely coupled with the core application.

3.8.1 Working of the Data Flow

The overall flow of information within the PrepSmart platform follows a sequential client-server communication model.

1. The user initiates an operation through the React web interface.
2. The frontend validates the request and forwards it to the Django REST API.
3. The backend authenticates the request using JWT tokens whenever authentication is required.

-
4. The requested business logic is executed within the appropriate backend module.
 5. Database operations are performed using Django ORM.
 6. When necessary, external services such as the Groq API, Kontests API, or the local code execution engine are invoked.
 7. The processed result is returned as a JSON response to the frontend.
 8. The frontend updates the user interface dynamically without reloading the page.

3.8.2 Advantages of the Data Flow Design

The proposed data flow architecture offers several advantages:

- Centralized processing of all client requests.
- Secure communication using authenticated REST APIs.
- Clear separation between presentation, business logic, and data management.
- Efficient interaction with external AI and coding services.
- Dynamic updating of user interfaces without full page refreshes.
- Scalable architecture suitable for future feature additions.

3.9 Class Diagram

A Class Diagram is one of the most important structural diagrams in the Unified Modeling Language (UML). It represents the static structure of the software by illustrating the classes, their attributes, and the relationships among them. The class diagram provides a blueprint for the implementation of the system and demonstrates how different software components collaborate to achieve the required functionality.

In PrepSmart, the class diagram is derived directly from the Django data models and business entities implemented during development. It represents the core classes responsible for user management, placement preparation, coding practice, assessments, interview management, career profile generation, and progress tracking.

Figure 3.6 illustrates the class diagram of the PrepSmart platform.

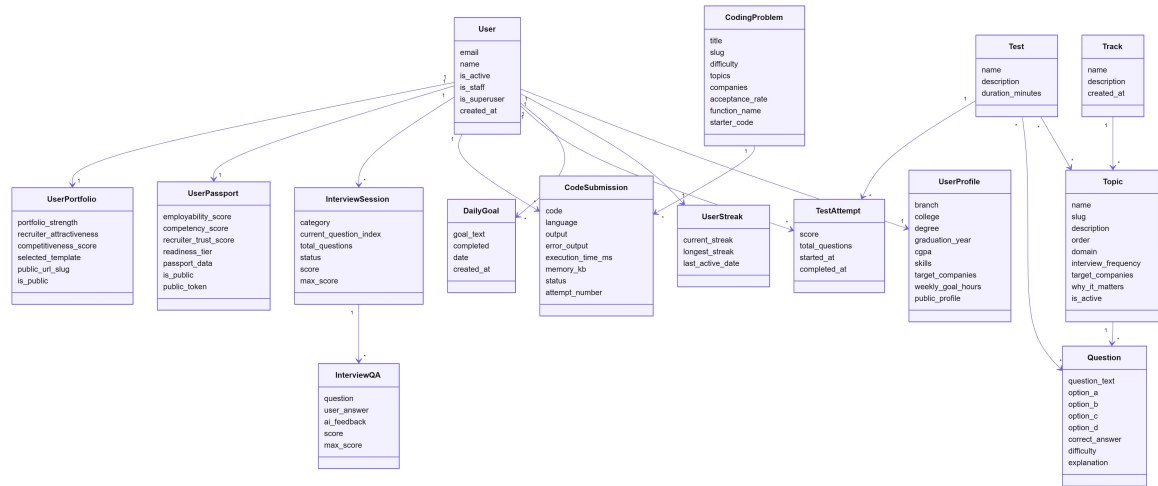


Figure 3.6: Class Diagram of PrepSmart

The class diagram is organized into multiple functional groups that collectively represent the object-oriented design of the PrepSmart platform. Each class encapsulates specific data and responsibilities while interacting with other classes through well-defined relationships.

The **User** class serves as the central entity of the system. It maintains authentication information and forms the foundation for almost every module within the application. Each registered user is associated with a single user profile, placement passport, portfolio, and learning statistics while maintaining one-to-many relationships with activities such as coding submissions, interview sessions, mock test attempts, and daily learning goals.

The **UserProfile** class stores academic and personal information including educational details, technical skills, target companies, placement goals, and user preferences. This information is utilized by various recommendation and analytical modules throughout the platform.

The learning management subsystem is represented through the **Track**, **Topic**, **Question**, and **Test** classes. Tracks organize placement preparation into multiple learning domains, while each topic contains educational resources, quizzes, and prerequisite relationships. Questions and tests evaluate student understanding and contribute to progress analysis.

The coding practice subsystem consists of the **CodingProblem** and **CodeSubmission** classes. CodingProblem stores programming challenges, test cases, difficulty levels, and related metadata, whereas CodeSubmission records the programs submitted by users, execution results, runtime statistics, memory usage, and submission status.

The interview preparation subsystem is modeled using the **InterviewSession** and **InterviewQA** classes. InterviewSession maintains the overall interview configuration

and progress, while InterviewQA records individual questions, user responses, AI-generated feedback, evaluation scores, and interview outcomes.

Career development features are represented through the **UserPortfolio** and **UserPassport** classes. These classes maintain employability metrics, project information, portfolio visibility, recruiter readiness indicators, and publicly shareable profile information. These modules assist students in presenting their skills and achievements effectively during placement activities.

The class diagram is organized into multiple functional groups that collectively represent the object-oriented design of the PrepSmart platform. Each class encapsulates specific data and responsibilities while interacting with other classes through well-defined relationships.

The **User** class serves as the central entity of the system. It maintains authentication information and forms the foundation for almost every module within the application. Each registered user is associated with a single user profile, placement passport, portfolio, and learning statistics while maintaining one-to-many relationships with activities such as coding submissions, interview sessions, mock test attempts, and daily learning goals.

The **UserProfile** class stores academic and personal information including educational details, technical skills, target companies, placement goals, and user preferences. This information is utilized by various recommendation and analytical modules throughout the platform.

The learning management subsystem is represented through the **Track**, **Topic**, **Question**, and **Test** classes. Tracks organize placement preparation into multiple learning domains, while each topic contains educational resources, quizzes, and prerequisite relationships. Questions and tests evaluate student understanding and contribute to progress analysis.

The coding practice subsystem consists of the **CodingProblem** and **CodeSubmission** classes. CodingProblem stores programming challenges, test cases, difficulty levels, and related metadata, whereas CodeSubmission records the programs submitted by users, execution results, runtime statistics, memory usage, and submission status.

The interview preparation subsystem is modeled using the **InterviewSession** and **InterviewQA** classes. InterviewSession maintains the overall interview configuration and progress, while InterviewQA records individual questions, user responses, AI-generated feedback, evaluation scores, and interview outcomes.

Career development features are represented through the **UserPortfolio** and **UserPassport** classes. These classes maintain employability metrics, project information, portfolio visibility, recruiter readiness indicators, and publicly shareable

profile information. These modules assist students in presenting their skills and achievements effectively during placement activities.

3.9.1 Advantages of the Class Design

The object-oriented class design adopted in PrepSmart provides several advantages:

- Promotes modular software development.
- Improves code reusability through well-defined classes.
- Simplifies maintenance and future feature additions.
- Supports scalability by separating different functional modules.
- Enhances readability and organization of the software architecture.
- Facilitates easier debugging and testing through modular implementation.

3.10 Database Design

The database is one of the most critical components of the PrepSmart platform, as it stores all information related to users, learning activities, coding practice, assessments, interview sessions, company preparation, portfolios, and analytical reports. A well-designed database ensures data consistency, minimizes redundancy, and enables efficient retrieval of information.

PrepSmart utilizes a relational database managed through Django Object Relational Mapping (ORM). Instead of directly writing SQL queries, Django ORM maps Python models to relational database tables, thereby simplifying database operations while improving portability and maintainability.

Figure 3.7 presents the Entity Relationship (ER) Diagram of the PrepSmart database.



Figure 3.7: Entity Relationship Diagram of PrepSmart

3.10.1 Major Database Entities

The PrepSmart database consists of multiple interconnected entities that collectively support the placement preparation ecosystem. The most important entities are described below.

- **User**

Stores authentication credentials and serves as the primary entity of the entire system. Every registered student is represented by a unique user record.

- **UserProfile**

Stores personal details, academic information, technical skills, CGPA, target companies, weekly study goals, and user preferences.

- **Track**

Represents different learning domains such as Data Structures, Algorithms, DBMS, Operating Systems, Aptitude, and Interview Preparation.

- **Topic**

Stores learning topics within each track together with descriptions, prerequisites, learning order, and interview importance.

- **Question**

Contains multiple-choice questions, explanations, difficulty levels, and correct answers used in quizzes and mock tests.

- **Test**

Represents mock tests and assessments that evaluate student preparation across multiple topics.

- **CodingProblem**

Stores coding challenges, supported programming languages, starter code, constraints, and evaluation criteria.

- **CodeSubmission**

Maintains submitted source code, execution results, runtime statistics, memory consumption, and submission status.

- **InterviewSession**

Stores AI interview sessions, interview categories, evaluation scores, and overall interview progress.

- **UserPortfolio**

Stores portfolio information, projects, public links, and recruiter-oriented presentation details.

3.10.2 Relationship Between Entities

The Entity Relationship Diagram illustrates several one-to-one and one-to-many relationships among the database entities.

The User entity acts as the central entity and maintains one-to-one relationships with UserProfile, UserPassport, UserPortfolio, and UserStreak. It also maintains one-to-many relationships with DailyGoal, TestAttempt, InterviewSession, CodeSubmission, UserAnswer, ActivityEvent, and CompanyTarget.

Each Track contains multiple Topics, while every Topic consists of multiple Questions, Topic Sections, and Learning Resources. Tests are associated with multiple Topics and Questions to facilitate comprehensive assessments.

Similarly, every CodingProblem can have multiple Test Cases and multiple Code Submissions, whereas every InterviewSession contains several Interview Questions and AI-generated evaluation records. These relationships ensure efficient organization of educational content while maintaining database normalization and referential integrity.

3.10.3 Database Features

The database design adopted in PrepSmart provides several advantages.

- Proper normalization reduces redundancy and improves storage efficiency.
- Foreign key relationships maintain referential integrity among entities.
- Django ORM provides database abstraction without requiring raw SQL queries.
- Modular entity design simplifies maintenance and future feature expansion.
- Indexed relationships improve query performance and dashboard generation.
- The schema supports scalability for additional learning modules, AI services, and company preparation features.

3.10.4 Advantages of the Database Design

The relational database design adopted in PrepSmart provides the following benefits.

- Efficient storage and retrieval of placement preparation data.
- Secure management of user information.
- Easy integration with Django REST Framework.
- Reduced data duplication through normalization.
- Improved scalability for future enhancements.
- Reliable transaction management using relational database principles.

3.11 Chapter Summary

This chapter presented the complete system analysis and design of the PrepSmart platform. It discussed the functional and non-functional requirements, feasibility analysis, overall system architecture, layered software architecture, high-level software organization, use case model, data flow, object-oriented class structure, and database design.

The presented UML diagrams and architectural models provide a comprehensive blueprint of the proposed system and illustrate the interactions among various software components. The modular architecture, well-structured database, and layered design ensure that the system is scalable, maintainable, secure, and capable of supporting future enhancements.

The next chapter describes the implementation of the proposed system, including frontend development, backend APIs, authentication mechanism, coding environment, AI interview module, dashboard analytics, and other core functionalities.

Chapter 4

System Development and Implementation

4.1 Introduction

This chapter describes the implementation of the proposed Smart Placement Preparation Platform (PrepSmart). Based on the system analysis and design presented in the previous chapter, the platform has been developed using modern web technologies following a modular client-server architecture.

The implementation focuses on providing an integrated environment for placement preparation by combining learning management, coding practice, mock assessments, AI-assisted interview preparation, company readiness analysis, portfolio management, and analytical dashboards within a single application.

The frontend of the platform has been developed using React and Vite, providing a responsive Single Page Application (SPA) with reusable components and dynamic user interfaces. The backend is implemented using Django and Django REST Framework (DRF), exposing secure RESTful APIs that communicate with the frontend through JSON-based requests and responses.

Authentication and authorization are implemented using JSON Web Tokens (JWT), ensuring secure access to protected resources. The application further integrates external services such as the Groq API for AI-powered interview assistance, the Kontests API for contest synchronization, and a local code execution environment for evaluating programming solutions.

This chapter presents the implementation details of the major software modules developed for PrepSmart, together with the corresponding workflows and interactions among various system components.

4.2 Technology Stack

The development of PrepSmart involved multiple technologies, frameworks, libraries, and development tools. Each technology was selected based on its performance, scalability, community support, and compatibility with the overall system architecture. The combination of these technologies enabled the development of a secure, modular, and interactive placement preparation platform.

Table 4.1 summarizes the technologies used during the development of the proposed system.

Table 4.1: Technology Stack Used in PrepSmart

Category	Technology	Purpose
Programming Language	Python	Backend development and business logic implementation.
Programming Language	JavaScript (ES6+)	Frontend development and client-side interactivity.
Frontend Framework	React (Vite)	Development of the Single Page Application (SPA).
Backend Framework	Django	Backend application development.
REST Framework	Django REST Framework (DRF)	Development of secure REST APIs.
Authentication	JSON Web Token (JWT)	Secure authentication and authorization.
Database	MySQL	Persistent storage of application data.
ORM	Django ORM	Database abstraction and model management.
AI Integration	Groq API	AI interview evaluation and intelligent explanations.
Contest Integration	Kontests API	Synchronization of coding contests.
Version Control	Git and GitHub	Source code management and collaboration.
API Testing	Postman	Testing REST API endpoints.
Development Environment	Visual Studio Code	Source code editing and debugging.
Code Execution	Local Subprocess Runtime	Compilation and execution of programming solutions.
Deployment Support	Django Static and Media Files	Management of static assets and uploaded content.

The selected technology stack provides a balanced combination of performance, scalability, maintainability, and ease of development. React enables the development of reusable user interface components, while Django and Django REST Framework provide a robust backend capable of handling authentication, business logic, and RESTful communication. Django ORM simplifies database interactions by mapping Python objects to relational database tables.

External integrations such as the Groq API and Kontests API extend the capabilities of the platform by providing AI-assisted interview functionality and real-time programming contest information. Development tools including Git, GitHub, Visual Studio Code, and Postman improve software development efficiency, version control, debugging, and API testing throughout the project lifecycle.

4.3 Development Environment

The development of PrepSmart was carried out using a modern full-stack web development environment. The project was implemented following a modular architecture with separate frontend and backend applications. Industry-standard development tools were used for coding, debugging, version control, API testing, and database management to ensure efficient software development.

The frontend application was developed using React with Vite, while the backend services were implemented using Django and Django REST Framework (DRF). Communication between the frontend and backend was achieved through RESTful APIs using JSON as the data exchange format. The database was managed using MySQL through Django ORM, providing secure and efficient data persistence.

Table 4.2 summarizes the software environment used during the implementation of PrepSmart.

Table 4.2: Development Environment of PrepSmart

Component	Software	Purpose
Operating System	Windows 11	Development and testing environment.
Code Editor	Visual Studio Code	Source code development and debugging.
Backend Language	Python 3.x	Backend programming.
Frontend Language	JavaScript (ES6+)	Client-side application development.
Backend Framework	Django	Server-side application development.
REST Framework	Django REST Framework	REST API development.
Frontend Framework	React (Vite)	Single Page Application development.
Database	MySQL	Data storage and management.
Version Control	Git	Source code version management.
Repository Hosting	GitHub	Source code hosting and collaboration.
API Testing	Postman	Testing backend REST APIs.
Web Browser	Google Chrome	Frontend testing and debugging.
Package Manager	npm	Frontend dependency management.
Backend Package Manager	pip	Python package management.

The selected development environment provided a reliable and efficient platform for implementing the proposed system. Visual Studio Code served as the primary Integrated Development Environment (IDE), offering debugging support, intelligent code completion, and extension-based development features.

Git and GitHub were used for source code version control, enabling systematic project management and tracking of code changes throughout the development life-cycle. Postman was extensively used to verify the correctness of REST API endpoints before integrating them with the React frontend.

The React development server provided rapid frontend updates during implementation, while Django’s built-in development server handled backend processing and API execution. MySQL, integrated through Django ORM, managed persistent application data efficiently without requiring manual SQL query implementation.

Overall, the chosen development environment supported rapid development, simplified debugging, improved code quality, and ensured smooth integration between different software components.

4.4 Frontend Implementation

The frontend of PrepSmart has been developed using React with Vite to provide a modern, responsive, and interactive Single Page Application (SPA). The frontend is responsible for presenting the user interface, handling user interactions, communicating with backend services through REST APIs, and dynamically updating application content without requiring complete page reloads.

The application follows a component-based architecture in which individual user interface elements are implemented as reusable React components. This modular approach improves maintainability, code reusability, and simplifies future feature enhancements.

Figure 4.1 illustrates the frontend architecture of the PrepSmart platform.

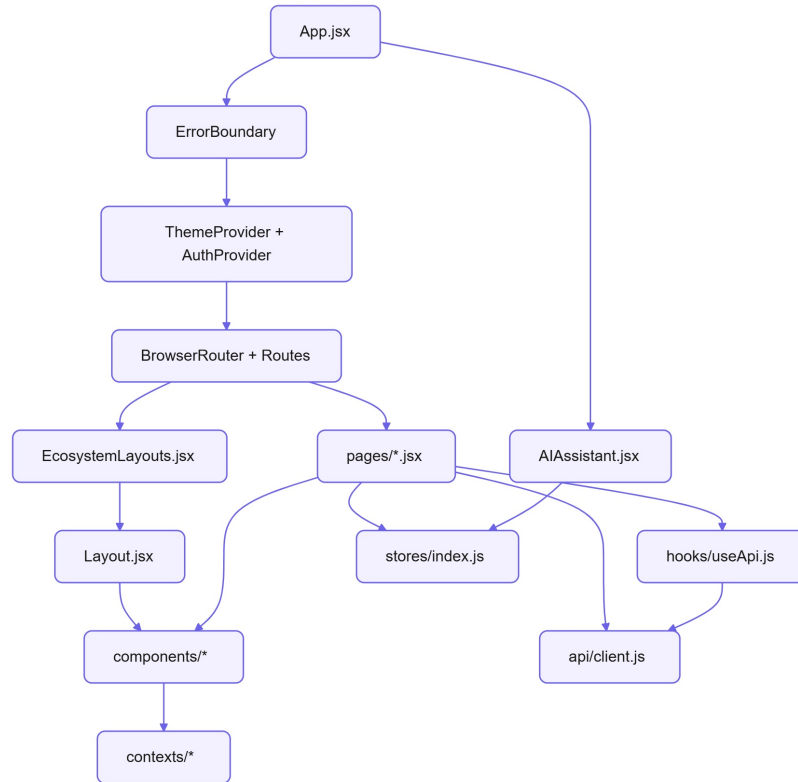


Figure 4.1: React Component Architecture of PrepSmart

The React application is organized around a hierarchical component structure that separates application initialization, routing, layout management, reusable components, and API communication into independent modules.

The application execution begins from the App component, which acts as the entry point of the frontend application. The App component initializes the global providers, configures routing, and renders the major modules of the application.

Global state management is handled through the ThemeProvider and AuthProvider components. ThemeProvider maintains the application's appearance settings, including light and dark themes, while AuthProvider manages authentication state, user information, access tokens, and session persistence throughout the application.

Navigation within the application is implemented using BrowserRouter. Instead of loading separate HTML pages, BrowserRouter dynamically renders React components according to the current URL, thereby providing a seamless Single Page Application experience.

The Layout component provides a consistent user interface across different pages by managing the navigation bar, sidebar, footer, and common interface elements. Individual pages such as Dashboard, Learning Modules, Coding Practice, Interview Preparation, Company Readiness, Portfolio Management, and Profile Management are rendered within this layout.

Reusable UI components are organized separately from page-level components. These reusable components include cards, buttons, forms, charts, dialogs, tables, progress indicators, and visualization widgets that are shared across multiple pages, reducing code duplication and improving consistency.

Communication with the backend is centralized through the API Client module. Instead of making HTTP requests directly from individual pages, all API interactions are performed through a common client that automatically attaches authentication tokens, processes server responses, and handles error conditions consistently.

Application-wide state management is further supported through React Contexts and Stores. These modules maintain shared application data such as authentication status, dashboard information, user preferences, and placement preparation progress without requiring excessive component-to-component communication.

The frontend also integrates an AI Assistant component that provides intelligent assistance and interacts with backend AI services whenever supported functionality is available.

4.4.1 Advantages of the Frontend Architecture

The frontend architecture adopted in PrepSmart offers several advantages.

- Component-based development improves code reusability.
- Single Page Application architecture provides faster navigation.

- Centralized API communication simplifies backend integration.
- Context Providers enable efficient global state management.
- Reusable UI components improve maintainability.
- Modular organization facilitates future feature expansion.
- Dynamic rendering enhances user experience without full page reloads.

4.5 Backend Implementation

The backend of PrepSmart has been developed using Django and Django REST Framework (DRF), following a modular architecture that separates authentication, business logic, database operations, and API communication into independent components. The backend serves as the central processing unit of the platform by handling client requests, validating user permissions, executing application logic, interacting with the database, and generating structured JSON responses.

RESTful APIs are used as the communication mechanism between the React frontend and the Django backend. Each client request passes through authentication and validation before the corresponding business logic is executed. This architecture improves security, maintainability, and scalability while supporting efficient interaction between the frontend and backend.

Figure 4.2 illustrates the request processing workflow adopted in PrepSmart.

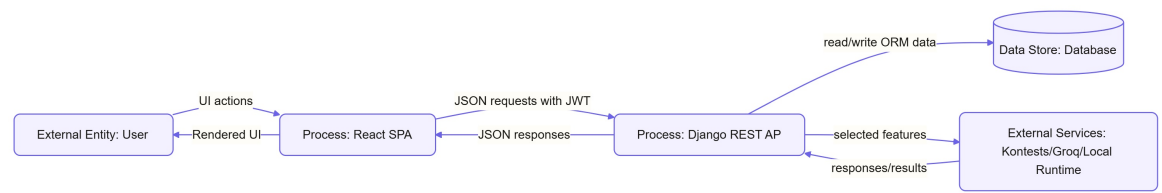


Figure 4.2: API Request Flow in PrepSmart

The API request processing begins when a user performs an action through the React frontend, such as logging into the system, starting a mock test, submitting code, or viewing dashboard statistics. The frontend sends an HTTP request through the centralized API Client using the appropriate REST endpoint.

The API Client automatically attaches the required HTTP headers, including the JSON Web Token (JWT) for authenticated requests. This ensures that protected resources are accessible only to authorized users.

The request is received by Django REST Framework, where authentication and permission checks are performed before forwarding the request to the appropriate

API View. Each API View is responsible for processing a specific functionality of the platform, such as user authentication, topic management, coding practice, interview sessions, or dashboard analytics.

The API View interacts with Django ORM to retrieve or update application data stored in the relational database. Business rules, validation logic, and data processing are executed within the backend before generating the response.

Once processing is complete, the backend returns a structured JSON response containing the requested information or the execution result. The React frontend receives this response and dynamically updates the user interface without requiring a complete page refresh, thereby providing a responsive and seamless user experience.

4.5.1 API Request Processing Workflow

The request handling mechanism followed by PrepSmart consists of the following sequential steps:

1. The user performs an operation through the React web interface.
2. The API Client generates an HTTP request and attaches the necessary authentication headers.
3. Django REST Framework authenticates the request using JSON Web Tokens (JWT).
4. The request is forwarded to the corresponding API View.
5. The API View executes the required business logic.
6. Django ORM performs the required database operations.
7. The processed result is converted into a JSON response.
8. The frontend receives the response and updates the user interface dynamically.

4.5.2 Advantages of the Backend Architecture

The backend implementation adopted in PrepSmart provides several advantages.

- Modular organization of backend services.
- Secure RESTful communication using JWT authentication.
- Efficient database interaction through Django ORM.

- Separation of business logic from presentation logic.
- Standardized JSON-based API communication.
- Improved scalability through independent backend modules.
- Easy integration with external APIs and AI services.

4.6 Authentication Module

Authentication is a fundamental component of the PrepSmart platform, ensuring that only authorized users can access protected resources and personalized placement preparation features. The system implements token-based authentication using JSON Web Tokens (JWT), providing a secure and scalable mechanism for user verification and session management.

The authentication module supports user registration, login, session validation, token refresh, and logout operations. Upon successful authentication, the backend generates an access token and a refresh token, which are securely managed by the frontend to maintain user sessions.

Figure 4.3 illustrates the authentication workflow implemented in PrepSmart.

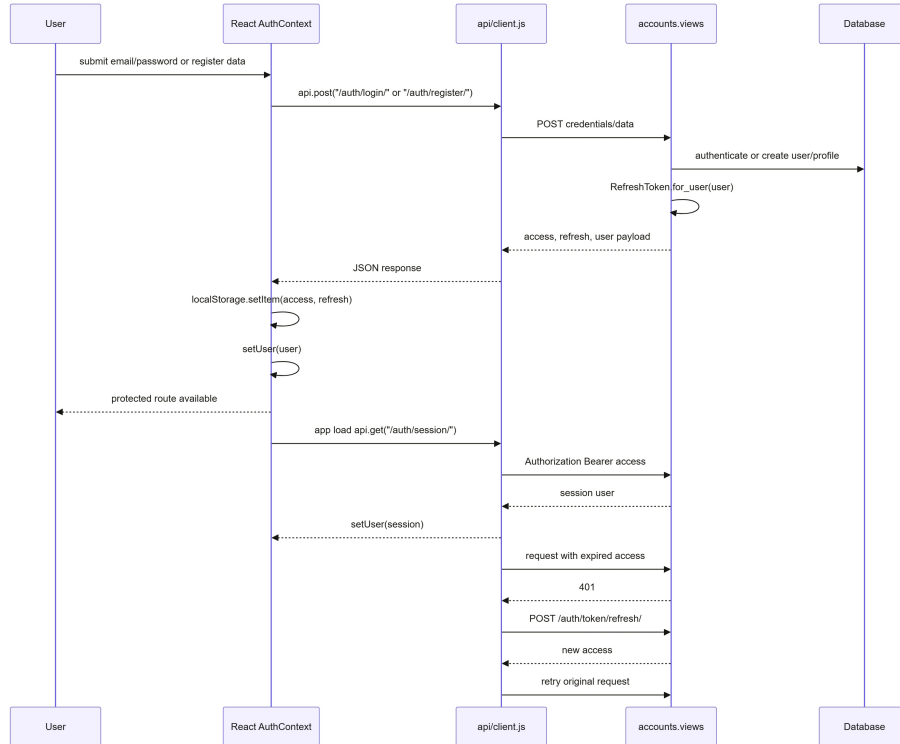


Figure 4.3: Authentication Sequence Diagram of PrepSmart

The authentication process begins when a user submits valid login credentials through the React frontend. The Auth Context forwards the credentials to the centralized API Client, which sends an authenticated HTTP POST request to the login endpoint exposed by Django REST Framework.

The backend verifies the supplied credentials against the stored user information in the database. If the credentials are valid, Django generates a JSON Web Token (JWT) pair consisting of an access token and a refresh token. Along with these tokens, the authenticated user's profile information is returned to the frontend.

The React application stores the received tokens securely in the browser's local storage and updates the authentication context with the logged-in user's information. Once authentication is successful, the user is redirected to the protected areas of the platform such as the dashboard, learning modules, coding workspace, and interview preparation pages.

Whenever the application starts, the frontend checks whether a valid user session already exists by requesting the session endpoint. If the access token is still valid, the backend returns the authenticated user's information and restores the session without requiring the user to log in again.

If an access token expires during API communication, the backend responds with an unauthorized status code. The API Client automatically sends the stored refresh token to the token refresh endpoint, receives a new access token, updates local storage, and retries the original request. This automatic refresh mechanism provides a seamless user experience while maintaining secure authentication throughout the application.

4.6.1 Authentication Workflow

The authentication mechanism implemented in PrepSmart follows the sequence below.

1. The user enters login credentials.
2. The frontend sends the credentials to the authentication API.
3. Django verifies the credentials using the user database.
4. A JWT access token and refresh token are generated.
5. Both tokens are returned to the frontend.
6. The frontend stores the tokens securely in local storage.
7. Protected API requests include the access token in the Authorization header.

-
8. If the access token expires, the refresh token is used to obtain a new access token.
 9. The original API request is automatically retried without requiring the user to log in again.

4.6.2 Security Features

The authentication module incorporates several security mechanisms to protect user accounts and application resources.

- Token-based authentication using JSON Web Tokens (JWT).
- Secure separation of access tokens and refresh tokens.
- Authentication of every protected API endpoint.
- Automatic token refresh without interrupting the user session.
- Prevention of unauthorized access through backend permission checks.
- Session restoration after successful token validation.
- Centralized authentication management using React Context.

4.6.3 Advantages of the Authentication Module

The implemented authentication mechanism provides several advantages.

- Secure user authentication.
- Stateless communication between frontend and backend.
- Reduced server-side session management.
- Improved scalability for multiple concurrent users.
- Seamless user experience through automatic token refresh.
- Simplified integration with RESTful APIs.

4.7 Placement Preparation Module

The Placement Preparation Module forms the core functionality of the PrepSmart platform. It provides students with a structured learning journey that organizes placement preparation into tracks, topics, quizzes, coding exercises, and progress monitoring. The objective of this module is to guide students through a systematic learning process instead of relying on fragmented preparation across multiple platforms.

The module continuously records user activities, evaluates learning progress, and updates completion status after every learning activity. This enables students to monitor their preparation while allowing the system to generate personalized recommendations and readiness analytics.

Figure 4.4 illustrates the workflow of the Placement Preparation Module.

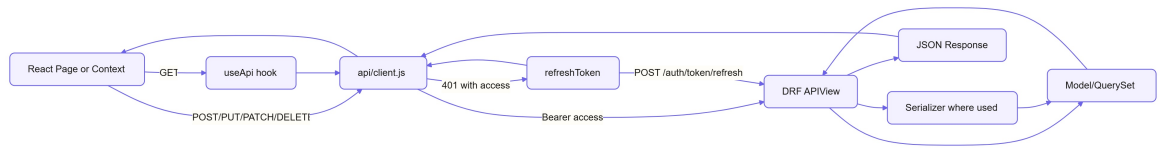


Figure 4.4: Placement Preparation Workflow

The placement preparation process begins when an authenticated student selects a learning track or topic from the frontend interface. The request is forwarded to the backend through the centralized API client, where the corresponding API endpoints process the request and retrieve the required learning content.

When a student completes a topic, the frontend sends a completion request to the backend. The backend verifies the authenticated user, retrieves the selected topic, and updates the corresponding learning progress record in the database. If the topic is being completed for the first time, the system creates a new progress entry and records the completion timestamp.

After completing the theoretical content, students attempt the topic quiz. Quiz responses are submitted to the backend, where each answer is evaluated against the stored correct answers. The backend calculates the student's accuracy and stores the result in the database. If the obtained score satisfies the predefined completion criteria, the corresponding topic is automatically marked as completed.

Every completed learning activity is recorded as an activity event, allowing the dashboard and analytics module to generate progress reports, calculate completion percentages, and recommend subsequent learning topics. This automated workflow ensures that students follow a structured placement preparation path while continuously tracking their overall readiness.

4.7.1 Placement Preparation Workflow

The Placement Preparation Module follows the workflow below.

1. The student selects a learning track.
2. The frontend requests the corresponding learning topics.
3. The backend retrieves topic information from the database.
4. The student studies the selected topic.
5. The student marks the topic as completed.
6. The backend updates the UserTopicProgress table.
7. The student attempts the topic quiz.
8. The backend evaluates quiz responses.
9. Progress statistics are updated.
10. Dashboard analytics are refreshed automatically.

4.7.2 Major Features

The Placement Preparation Module provides the following features.

- Structured learning tracks.
- Topic-wise study materials.
- Quiz-based assessment.
- Automatic progress tracking.
- Activity logging.
- Personalized learning journey.
- Readiness score calculation.
- Integration with dashboard analytics.

4.7.3 Advantages of the Placement Preparation Module

The implemented module offers several advantages.

- Organizes placement preparation into structured learning paths.
- Eliminates fragmented learning across multiple platforms.
- Automatically tracks student progress.
- Provides continuous assessment through quizzes.
- Enables personalized learning recommendations.
- Supports future expansion with additional learning tracks.

References

- [1] React Documentation, Meta Platforms Inc. [Online]. Available: <https://react.dev/>
- [2] Django Documentation, Django Software Foundation. [Online]. Available: <https://docs.djangoproject.com/>
- [3] Django REST Framework Documentation. [Online]. Available: <https://www.django-rest-framework.org/>
- [4] MySQL Documentation, Oracle Corporation. [Online]. Available: <https://dev.mysql.com/doc/>
- [5] JSON Web Token (JWT) Introduction. [Online]. Available: <https://jwt.io/introduction/>
- [6] MDN Web Docs (HTML, CSS, JavaScript). [Online]. Available: <https://developer.mozilla.org/>
- [7] Python Documentation. [Online]. Available: <https://docs.python.org/3/>
- [8] GitHub Repository (PrepSmart Project). [Online]. Available: <https://github.com/neeraj-cseh/Project>
- [9] Stack Overflow Developer Community. [Online]. Available: <https://stackoverflow.com/>
- [10] Postman API Documentation. [Online]. Available: <https://learning.postman.com/>
- [11] ChatGPT, OpenAI. [Online]. Available: <https://chat.openai.com/>
- [12] OpenAI Codex, OpenAI. [Online]. Available: <https://chatgpt.com/codex/>
- [13] Gemini AI, Google DeepMind. [Online]. Available: <https://gemini.google.com/>
- [14] Claude AI, Anthropic. [Online]. Available: <https://claude.ai/>

[15] YouTube Tutorials (Various Channels). [Online]. Available: <https://www.youtube.com/>